Homework 2

刘滨瑞　2021012579　未央-水木 12

2024 年 10 月 16 日

1

临界底坡（Critical slope），即在某一流量条件下，正常水深 h_0 恰好等于临界水深 h_c 的底坡。

当渠道的底坡大于临界底坡时，为陡坡（Steep slope），此时正常水深小于临界水深。当渠道的底坡小于临界底坡时，为缓坡（Mild slope），此时正常水深大于临界水深。

2

比能沿程增加，即 $\frac{\mathrm{d}E_s}{\mathrm{d}s} > 0$，这等价于 $i > J_f$，进而等价于 $h > h_0$。

因此，在 12 条水面曲线中，M_1，S_1，S_2，C_1 满足条件，是比能沿程增加的水面曲线。

3

3.1

对于临界水深，有：$\frac{\alpha Q}{g} = \frac{A_c^3}{B_c^3} = \frac{(bh_c)^3}{b}$。

取 $\alpha = 1$，求得：$h_c = Q^{\frac{2}{3}} g^{-\frac{1}{3}} b^{-\frac{2}{3}} = 0.861m$。

对于临界底坡，有：$\frac{Q}{\sqrt{i_c}} = K_c = \frac{A_c}{n} R^{\frac{2}{3}} = (bh_c)^{\frac{5}{3}} (b + 2h_c)^{-\frac{2}{3}} n^{-1}$。

求得：$i_c = Q^2 (b + 2h_c)^{\frac{4}{3}} n^2 (bh_c)^{-\frac{10}{3}} = 5.301 \times 10^{-3}$。

注意到 $i_c > i_1 > i_2$，因此两段渠道都是缓坡。

3.2

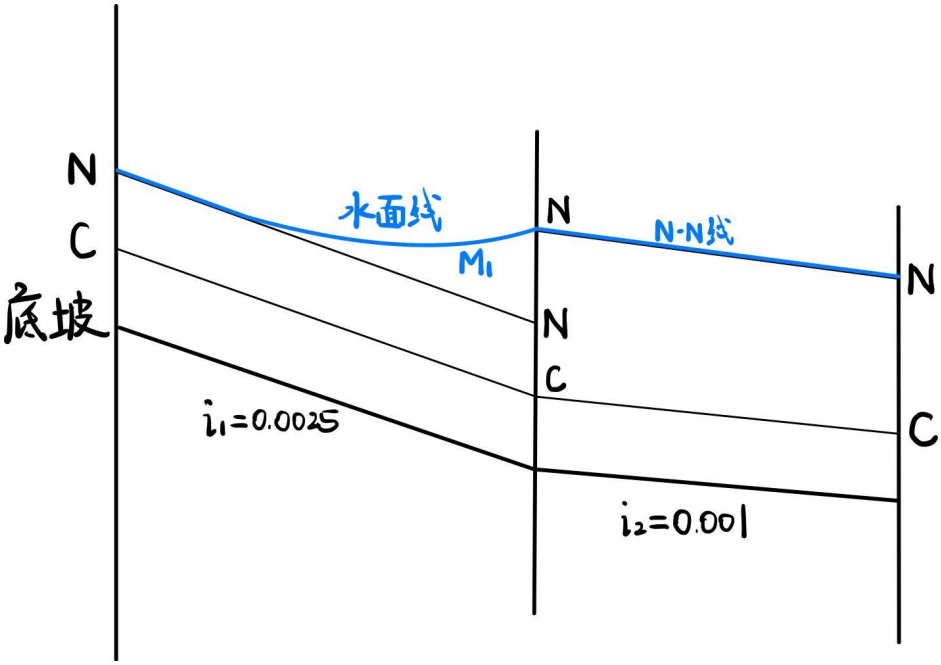


图 1: 水面线型式

4

从第一段的缓坡过渡到第二段的陡坡，水面会出现水跌，因此水面线会经过临界水深线，可以认为 $h_c = 3m$。

取 $\alpha = 1$，对于矩形渠道：$q = \sqrt{gh_c^3} = 16.27m^2s^{-1}$。

5

5.1

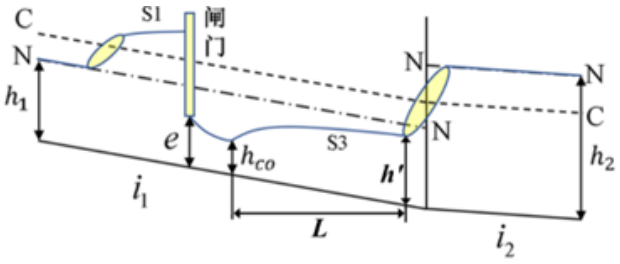


图 2: 题 5 图

基本公式:

$$\Delta s = \frac{E_{sd} - E_{su}}{i - \overline{J_f}}$$

若认为缩断面后到水跃前的流动整体为渐变流，有:

$$L = \frac{(h' - h_{co}) + \frac{\alpha}{2g}(\frac{q^2}{h'^2} - \frac{q^2}{h_{co}^2})}{i_1 - \overline{J_f}}$$

假定 $b \gg h$，流量模数:

$$K = \frac{bh}{n}(\frac{bh}{b+2h})^{\frac{2}{3}} = \frac{bh^{\frac{5}{3}}}{n}$$

损失为:

$$J_f = \frac{(qb)^2}{K^2} = \frac{q^2n^2}{h^{\frac{10}{3}}}$$

对于 $\overline{J_f}$，我们采用几何平均值的计算方法:

$$\overline{J_f} = \sqrt{\frac{q^2n^2}{h'^{\frac{10}{3}}}\frac{q^2n^2}{h_{co}^{\frac{10}{3}}}} = \frac{(qn)^2}{(h'h_{co})^{\frac{5}{3}}}$$

代入数值，计算得:

$$\overline{J_f} = 0.03960$$

$$L = 77.99m$$

5.2

沿用 5.1 中的计算方式，将研究流域等分为 N 段进行计算。

计算代码如下：

```
from numpy import linspace, power, log2
import matplotlib.pyplot as plt

q = 5.61
h_co = 0.62
h_p = 1.06
n = 0.025
i1 = 0.01
alpha = 1.0
g = 9.8

def J(h1, h2):
    return power((q * n), 2) \
            / power((h1 * h2), (5/3))

def s(h1, h2):
    return ((h2 - h1) + \
            alpha * power(q, 2) / (2 * g) \
            * (power(h2, -2) - power(h1, -2))) \
            / (i1 - J(h2, h1))

Ns = [1, 2, 3, 4, 6, 10,20, 40, \
        100, 200, 400, 1000, 4000, \
        20000, 100000] # Number of segments
Ls = []
for N in Ns:
    hs = linspace(h_co, h_p, N + 1)
```

```
    re0 = 0
    for i in range(N):
        re0 += s(hs[i], hs[i + 1])
    Ls.append(re0)

print(f"L = {re0} m")

plt.figure()
plt.title("log(N)-L")
plt.plot(log2(Ns), Ls, marker='o')
plt.xlabel("log(N)")
plt.ylabel("L")
plt.savefig("Ls.png")
```

如图 3 所示，随着 N 的增长，L 逐渐收敛，计算结果为：

$$L = 81.32m$$

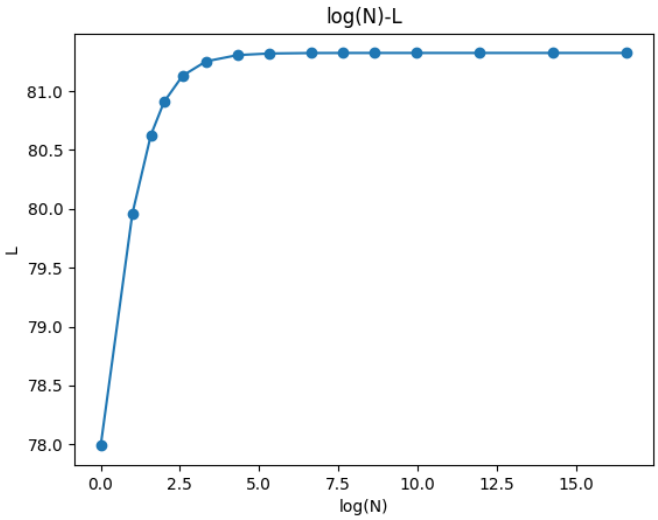


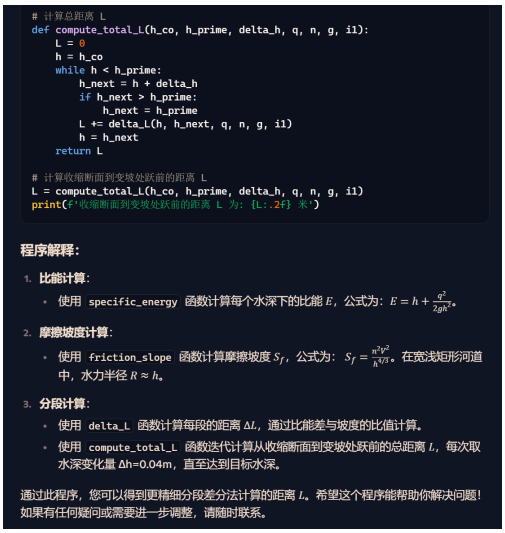
图 3: 算法收敛情况

图 3 表明，我们的数值算法计算平稳，收敛性良好。该算法等价于对 $\frac{\mathrm{d}L}{\mathrm{d}s}$ 进行积分，因此其收敛结果应为 L 真实值。

该方法得到的答案略大于整段计算得到的答案，相对误差为 4.1%，这说明我们在 5.1 中使用的渐变流假设是可取的，所造成的误差并不大。

使用 New Bing。对话记录参见图 4。

(a) Part 1



(b) Part 2

图 4: 与 New Bing 的对话记录

研究 New Bing 给出的代码，其解答思路基本正确，仅有一个小 bug：25 行的摩擦坡度前缺少一个负号。在修改后，代码运行结果为 $L = 81.27m$，与我们的答案一致，仅差 $0.05m$。